

numaring (NumaRing)

Comprehensive Technical Engineering & Research Roadmap

Phase-by-Phase Development, Verification, & Paper Drafting Roadmap | Engineering Plan

[ENGINEERING ROADMAP OVERVIEW]

This roadmap outlines the complete development cycle for numaring across 6 structured phases: from repository architecture and NUMA topology discovery to core lock-free data structure development, batched work-stealing, hardware-level profiling, and paper composition.

1. Roadmap Executive Summary

Phase	Focus Area	Key Deliverables
Phase 1	Environment & Architecture Setup	Modern CMake setup, C++20 build target, libnuma linking, 128-byte cache alignment wrappers.
Phase 2	Topology Discovery Engine	Hardware NUMA topology detection via libnuma, core-to-node routing via sched_getcpu(), fallback pathways.
Phase 3	Node-Local Ring Buffer Core	Bounded lock-free SPSC/MPMC ring buffers with acquire-release atomics and software prefetching hints.
Phase 4	Batched Cross-Node Work Stealing	Inter-node chunk transfers (16-32 items), underflow/overflow handling, cross-socket atomic minimization.
Phase 5	Microarchitectural Profiling Suite	Google Benchmark harness, tail latency profiling (p99/p99.9), perf c2c cache invalidation & numastat metrics.
Phase 6	Paper Writing & Publication	Formulating paper methodology, benchmark visualization graphs, USENIX ATC / PPoPP manuscript drafting.

2. Detailed Phase Breakdowns

Phase 1: Project Scaffolding & Memory Alignment Primitives

- **Repository Structure Setup:** Establish clean header-only directory layout (``include/numaring/``) alongside Google Benchmark (``benchmarks/``) and Google Test (``tests/``) harnesses.
- **Build System Configuration:** Create CMake build scripts targeting C++20 standard (``-std=c++20``), enabling optimization flags (``-O3``, ``-march=native``), and linking ``libnuma`` (``-lnuma``).
- **Hardware Cache Alignment Wrapper:** Design strict 128-byte dual-cache-line padding structures using ``alignas(128)`` to isolate atomic variables from L2 spatial prefetcher false sharing.

Phase 2: NUMA Topology Discovery & Dynamic Routing

- **Hardware Topology Detection:** Interface with ``libnuma`` system calls (``numa_available()``, ``numa_max_node()``) to inspect available CPU sockets and memory nodes at runtime.
- **Core-to-Node Mapper:** Implement lightweight thread routing using ``sched_getcpu()`` and ``numa_node_of_cpu()`` to identify executing NUMA domains with minimal instruction overhead.
- **Fallback Mechanism:** Provide a graceful degradation pathway to a single-node MPMC ring buffer when running on unified or single-socket architectures.

Phase 3: Core Node-Local Ring Buffer Development

- **Node-Local Queue Memory Allocation:** Allocate ring buffer arrays directly on target NUMA memory controllers using ``numa_alloc_onnode()`` to ensure zero remote DRAM access during local queue operations.
- **Lock-Free Atomic State Engine:** Implement bounded ring buffer slot sequence tracking using fine-grained acquire-release atomics (``std::memory_order_acquire``, ``std::memory_order_release``).
- **Software Prefetch Integration:** Insert explicit prefetching instructions (``_mm_prefetch``) to pre-load target slot data addresses into L1 data caches prior to consumer access.

Phase 4: Batched Inter-Node Work Stealing & Load Balancing

- **Overflow/Underflow Detection:** Implement fast local capacity checks to trigger cross-node synchronization only when a local queue fills up or runs dry.
- **Batched Chunk Transfers:** Develop atomic block transfer primitives that move items in contiguous chunks (16–32 items) across NUMA nodes in a single atomic update.
- **Interconnect Contention Reduction:** Verify that batched transfers reduce inter-socket atomic CAS operations by up to 93.75% compared to single-item cross-socket accesses.

Phase 5: Microarchitectural Profiling & Benchmarking

- **Throughput & Latency Suite:** Build Google Benchmark programs to measure throughput (ops/sec) and tail latency (p95, p99, p99.9) across thread scaling from 1 to 32+ cores.

- **Baseline Comparative Analysis:** Benchmark numaring against standard industry baselines (``boost::lockfree::queue``, `moodycamel `ConcurrentQueue``, and mutex queues).
- **Low-Level Hardware Metrics:** Execute ``perf c2c`` to record cross-socket HITM cache line invalidations and ``numastat`` to confirm > 98% local NUMA memory hit ratios.

Phase 6: Paper Composition & Publication Preparation

- **Manuscript Drafting:** Compose full research paper detailing microarchitectural motivation, system design, formal queue semantics, and benchmark evaluation.
- **Benchmark Visualization:** Generate clear performance charts illustrating throughput scaling curves, tail latency distributions, and cache invalidation reductions.
- **Artifact Open-Sourcing:** Package numaring repository as a fully reproducible artifact for review at systems venues (e.g., USENIX ATC, PPOPP, or IEEE TPDS).